The new SATA, step by step

from the input (the channel) to the output (the corrected channel), in the order the code runs — and the difference between whole band and per sub-band

Lucas Ayub — SATA 1D verification

September 2026

How to read this document

SATA has **two stages**, always in this order:

1. **Before the loop** (once per channel): build the **map** $\Delta C_0[x]$ — a table that says *which correction each ground position* x *needs*.
2. **The loop** (once per window): cut a window of the signal, take the FFT, find out *which position* each bin points to, **read the map at that position** and rotate the phase of the bin.

Each section below is one step, in execution order. For each step: **why** it exists, **the maths** built from the physics, **the code**, and **the numbers** of one example that we follow from start to end.

The example followed through the whole document

One target at $h = 240$ m, at the same slant range as the scene centre ($r_0 = 766.2$ km). Array of the IRF test: $N_{rx} = 4$, PRF = 2000 Hz, $\mathrm{PRF}_{op} = 500$ Hz per channel, b_{at} on DPCA (7.69 m), $b_{xt} = [63.7,\ 27.0,\ 4.1,\ 1.7]$ m. We follow **channel 0** ($b_{xt} = 63.7$ m). Constants: $\lambda = 0.25$ m, $v = 7688.5$ m/s.

Contents

1 The overall flowchart **2**

2 Step 0 — The input: the channel **3**

3 Preliminary step — The ruler: from frequency to position **4**

4 Stage A — Build the map (before the loop) **5**

4.1 A1 — How much each target must be corrected: ΔC_0 5

4.2 A2 — Where the target sits on the channel axis 6

4.3 A3 — How wide the target appears in the STFT: hw 6

4.4 A4 — Where else the target appears: the alias images 7

4.5 A5 — Draw the map 7

5 Stage B — The loop (once per window) **8**

5.1 One window, with numbers (centre $c = 1510$) . . . 9
5.2 The same target across the windows . . . 10
5.3 B6 — Putting the windows back together (overlap-add) . . . 10

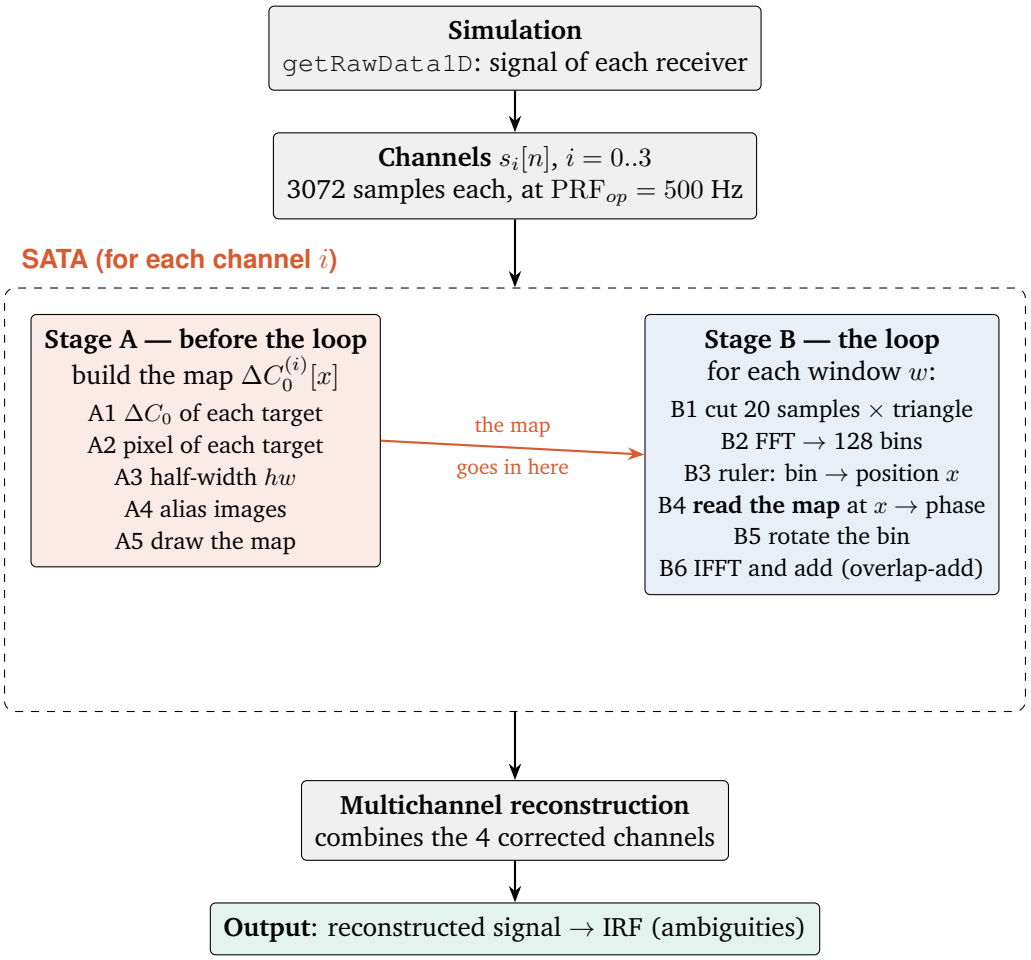
6 Stage C — The reconstruction (what happens next) 11

7 Whole band $\times$ per sub-band 12
7.1 The two flows . . . 12
7.2 The three differences, one by one . . . 12
7.3 Why, here, both give the same result . . . 13
7.4 When per sub-band would make a difference . . . 13

8 The result 14

9 Summary: the order of execution 14

1 The overall flowchart



Where this happens in the code

```
# whole band  (sar_recon/sata.py)
def sata_channels(cfg, tracks, s_channel, remove=True, sata_osf=4):
    for kk in range(Nrx):                                    # for each
    channel
        dC0 = build_delta_C0_array(cfg, tracks, kk)           # STAGE A (
    the map)
        out[kk] = sata_1d(out[kk], dC0, ..., inverse=remove) # STAGE B (
    the loop)
    return out
rec = reconstruct(cfg, tracks, sata_channels(...))           #
    reconstruction
```

Per sub-band has the **same** structure, repeated once per sub-band (Section 7).

What changed in the "new SATA"

Only **Stage A**. The loop (Stage B, `sata_1d`) is the same as before. Before, the map was the target's ΔC_0 *repeated over the whole line*; now ΔC_0 is present **only where each target appears in the STFT** and zero elsewhere.

2 Step 0 — The input: the channel

The channel axis

Slow time is centred: the radar is at $x = 0$ halfway through the acquisition. The reconstruction works at 2000 Hz, but each receiver keeps 1 of every 4 pulses, i.e. $\mathrm{PRF}_{op} = 500$ Hz. Between two channel samples the satellite moves

$$\Delta s = \frac{v}{\mathrm{PRF}_{op}} = \frac{7688.5}{500} = 15.4 \text{ m}.$$

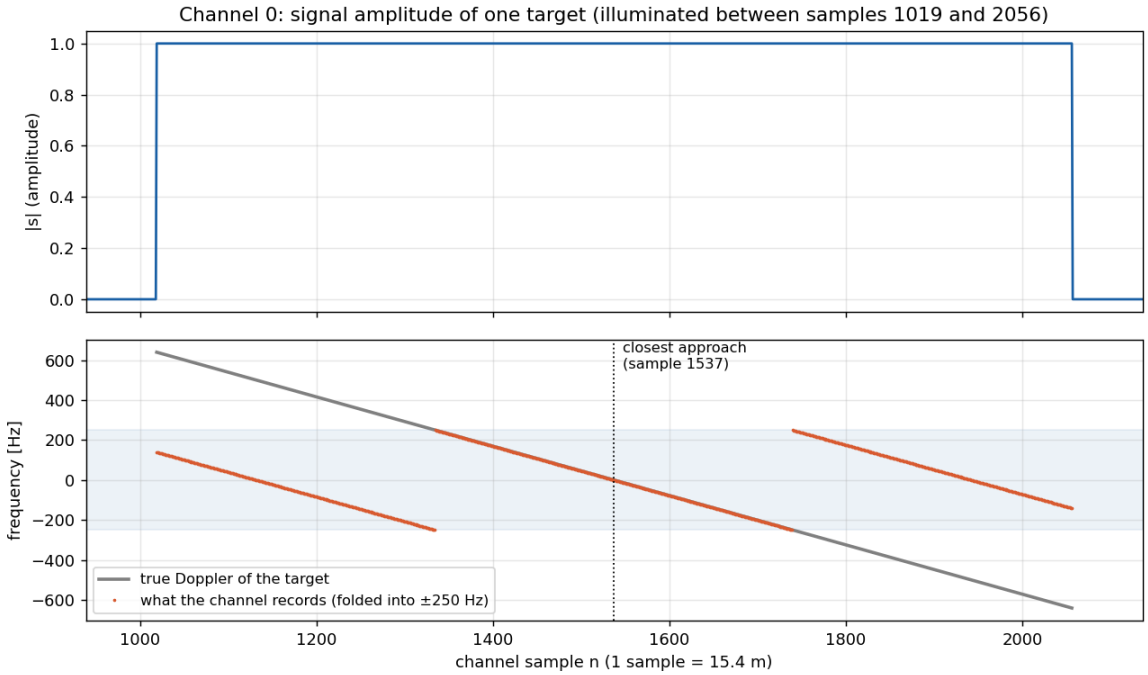
So **"channel sample" and "position along the track" are the same thing**: sample n corresponds to the radar at $x = (n - N_{a,ch}/2)\,\Delta s$.

The signal of one target

For each pulse the simulator computes the path Tx→target→Rx, $rh(t) = |p_{tx} - p| + |p_{rx} - p|$, and records $s(t) = e^{-j2\pi\, rh(t)/\lambda}$ while the target is inside the beam. The phase changes because rh changes; the **instantaneous frequency** (Doppler) is the derivative of the phase:

$$f_D(t) = \frac{1}{2\pi}\frac{d}{dt}\Big(-\frac{2\pi}{\lambda} rh(t)\Big) = -\frac{1}{\lambda}\frac{d\,rh}{dt}.$$

While the radar approaches the target, rh decreases and $f_D > 0$; at closest approach $f_D = 0$; afterwards $f_D < 0$.



Top: the target is only illuminated between samples 1019 and 2056 (the antenna beam). Bottom: grey is the true Doppler, from $+640$ to -640 Hz; orange is what the channel actually records.

Why the orange line "jumps": aliasing

Sampling at 500 Hz, the channel can only tell apart frequencies between -250 and $+250$ Hz (blue band). A frequency outside that range is recorded as $f - m \cdot 500$ Hz (whichever falls inside the band). For example, $+527$ Hz shows up as $+27$ Hz. **The target's signal is in the channel the whole time — only, at the ends of the aperture, with a "folded" frequency.** This will matter in Step A4.

3 Preliminary step — The ruler: from frequency to position

Before building the map we need the tool the loop uses to *read* it: the ruler that turns the frequency of a bin into a ground position. It is used in Steps A3, A4 and B3.

Building the ruler

(1) Frequency → angle. The radar moves at speed v. A target seen at an angle θ from "straight to the side" approaches at speed $v \sin\theta$. The round-trip path shortens by $2v \sin\theta$ per second, hence

$$f = -\frac{1}{\lambda}\frac{d\,rh}{dt} = \frac{2v\sin\theta}{\lambda} \qquad \Longleftrightarrow \qquad \boxed{\theta = \arcsin\frac{\lambda f}{2v}}$$

(2) Angle → ground distance. If the radar is at x_{radar} and looks at angle θ at distance r, the point it sees is ahead by

$$x - x_{\text{radar}} = r\tan\theta \quad \text{[m]}.$$

(3) Metres → cells. Dividing by $\Delta s = v/\mathrm{PRF}_{op}$:

$$\boxed{\text{azpos}(f) = \frac{r\,\tan\left(\arcsin\frac{\lambda f}{2v}\right)}{v}\,\mathrm{PRF}_{op}} \qquad \text{[cells]}.$$

In a window whose centre (the radar position) is sample c, the bin of frequency f_k points to the cell

$$\boxed{\texttt{posaux}_k = c + \text{azpos}(f_k)}$$

```
f       = bin_frequencies(Nzp)                    # 128 bins in [-250,
    250) Hz
betasub = np.arcsin(wl * f / (2 * v))             # (1) f -> angle
azpos   = r * np.tan(betasub) / v * prf           # (2)+(3) angle ->
    cells
posaux  = np.round(azpos + center).astype(int)    # + radar position
```

Numbers of the ruler

Since the angles are tiny (< 0.25), the ruler is practically a straight line: $\text{azpos}(f) \approx \frac{r\lambda\,\mathrm{PRF}_{op}}{2v^2} f = 0.81$ cells/Hz.

	f	θ	azpos
1 bin (128 bins over 500 Hz)	3.9 Hz	0.002°	3.2 cells
edge of the band	±250 Hz	±0.23	±203 cells

Each window "sees" the ground from $c - 203$ to $c + 203$ cells (±3.1 km).

The ruler and aliasing

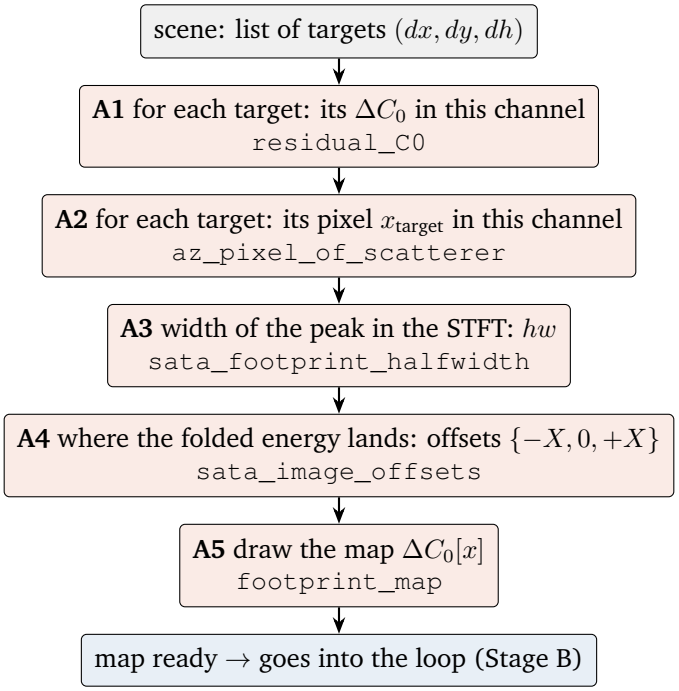
A bin does not know whether its frequency is f or $f \pm 500$ Hz. If the energy came from f_{true} but was recorded at $f_{\text{true}} - m \cdot 500$, the ruler sends it to

$$c + \text{azpos}(f_{\text{true}} - m\,\mathrm{PRF}_{op}) \approx x_{\text{target}} - m\,X,$$

$$X = \text{azpos}(\mathrm{PRF}_{op}) = \frac{r\,\tan\left(\arcsin\frac{\lambda\mathrm{PRF}_{op}}{2v}\right)}{v}\mathrm{PRF}_{op} = 405 \text{ cells}.$$

In words: **energy folded once always lands 405 cells away from the target**, in any window.

4 Stage A — Build the map (before the loop)



All of this is inside the function `build_delta_C0_array` (file `sar_recon/sata.py`) and runs **once per channel**, before the loop.

4.1 A1 — How much each target must be corrected: ΔC_0

Why. The reconstruction was designed for a target on flat ground (the scene centre, $h_0 = 0$). A target with height, seen by a receiver displaced sideways (b_{xt}), travels a slightly different path from what the reconstruction assumes. That difference is what we need to remove.

The maths, built up. The pipeline describes the path of each channel with a parabola in time (measured from closest approach):

$$\underbrace{rh_{MS}(t)}_{\text{Tx}\to\text{target}\to\text{Tx}} \approx c_0 + c_1 t + c_2 t^2, \qquad \underbrace{rh_{BS}(t) - rh_{MS}(t)}_{\text{extra part of the bistatic channel}} \approx \Delta c_0 + \Delta c_1 t + \Delta c_2 t^2.$$

Taking this to the frequency domain (stationary phase), each parabola becomes a constant, a linear and a quadratic term in f. The **constant** term of the difference is C_0:

$$C_0 = \Delta c_0 + \frac{c_1^2}{4c_2} - \frac{(c_1 + \Delta c_1)^2}{4(c_2 + \Delta c_2)} \approx \Delta c_0 \quad \text{(the extra path at closest approach).}$$

What matters is how much the target's C_0 differs from the C_0 the reconstruction assumes:

$$\boxed{\Delta C_0^{(i)} = C_0^{(i)}(\text{real target}) - C_0^{(i)}(\text{flat centre})} \quad \Rightarrow \quad \text{phase error in channel } i = \frac{2\pi}{\lambda}\,\Delta C_0^{(i)}.$$

```
def residual_C0(cfg, tracks, ptg_real, channel):
    C0_real = GetCoeffNu(ptg_real,      ...channel...)[0]   # parabola fit,
   real target
    C0_ref  = GetCoeffNu(cfg.scene.ptg, ...channel...)[0]   # same, flat centre
    return C0_real - C0_ref
```

In the example

channel	0	1	2	3
b_{xt} [m]	63.7	27.0	4.1	1.7
ΔC_0 [cm]	-5.474	-2.319	-0.352	-0.142
phase $\frac{2\pi}{\lambda}\Delta C_0$	-78.8	-33.4	-5.1	-2.0

The larger the sideways displacement b_{xt}, the larger the error. We follow channel 0: $\Delta C_0 = -5.474$ **cm**.

4.2 A2 — Where the target sits on the channel axis

Why. The map is indexed by position; we need to know in which cell to put ΔC_0.

The maths. From Step 0, sample $N_{a,ch}/2$ corresponds to $x = 0$ and each sample is Δs metres. The target is at $x_0 + dx$. But receiver i trails the transmitter by $b_{at,i}$, and the "eye" of the Tx–Rx pair is halfway between them, $b_{at,i}/2$ behind; so it passes the target slightly later:

$$\boxed{x^{(i)}_{\text{target}} = \frac{N_{a,ch}}{2} + \frac{x_0 + dx + b_{at,i}/2}{\Delta s}}$$

```
def az_pixel_of_scatterer(cfg, dx, channel):
    ds    = cfg.system.vs / cfg.PRF_op
    x_abs = cfg.scene.x0 + dx + 0.5 * cfg.array.bat[channel]
    return int(round(cfg.Na_ch / 2 + x_abs / ds))
```

In the example: $N_{a,ch}/2 = 1536$, $x_0 = 20$ m, $dx = 0$, $b_{at,0} = 0 \Rightarrow x_{\text{target}} = 1536 + 20/15.4 =$ **1537** (the closest approach in the Step 0 figure).

4.3 A3 — How wide the target appears in the STFT: hw

Why. The loop looks at the signal through windows of only 20 samples. A short window cannot resolve fine frequencies: a target does not become a "line" in a single bin, it becomes a **hill** spread over several bins, hence several cells. ΔC_0 must cover the whole hill, otherwise part of the target's energy is not corrected.

The maths, built up.

1. The window has $T_{sub,eff} = 20$ samples and is multiplied by a triangle. The FFT of a triangle is a sinc^2: main lobe with the 1st null at $2\text{PRF}_{op}/T_{sub,eff} = 50$ Hz, first sidelobes (at -26.5 dB) up to the 2nd null at $4\text{PRF}_{op}/T_{sub,eff} = 100$ Hz.
2. We want to cover everything up to the 2nd null (everything above ~ -30 dB).
3. Passing 100 Hz through the ruler:

$$hw = \text{azpos}(100\text{ Hz}) = \frac{r \tan\left(\arcsin\frac{\lambda \cdot 100}{2v}\right)}{v}\,\text{PRF}_{op} \approx \mathbf{82}\text{ cells } (\approx 1.26\text{ km}).$$

```
def sata_footprint_halfwidth(cfg):
    Tsubeff = round(sqrt(wl*r/2) * prf / v * 0.5) * 2          # 20, same as
   sata_1d
    f_edge  = 4.0 * prf / Tsubeff                              # 2nd null: 100
   Hz
    return ceil(r * tan(arcsin(wl*f_edge/(2*v))) / v * prf)    # -> 82 cells
```

4.4 A4 — Where else the target appears: the alias images

Why. From Step 0, at the ends of the aperture the target's Doppler exceeds ± 250 Hz and shows up folded. Through the ruler (Section 3), that energy lands **405 cells away** from the target. It is still the target's energy and also needs the target's ΔC_0.

The maths. The target's Doppler goes up to $\pm B/2$, with $B = 4v\sin(\theta_{tx}/2)/\lambda = 1281$ Hz (the beamwidth). It folds $m = \mathrm{round}(f/\mathrm{PRF}_{op})$ times, i.e. $m \in \{-1, 0, +1\}$ (640 Hz $<$ 750 Hz). Offsets:

$$\{\, m \cdot X \,\} = \{-405,\ 0,\ +405\} \text{ cells.}$$

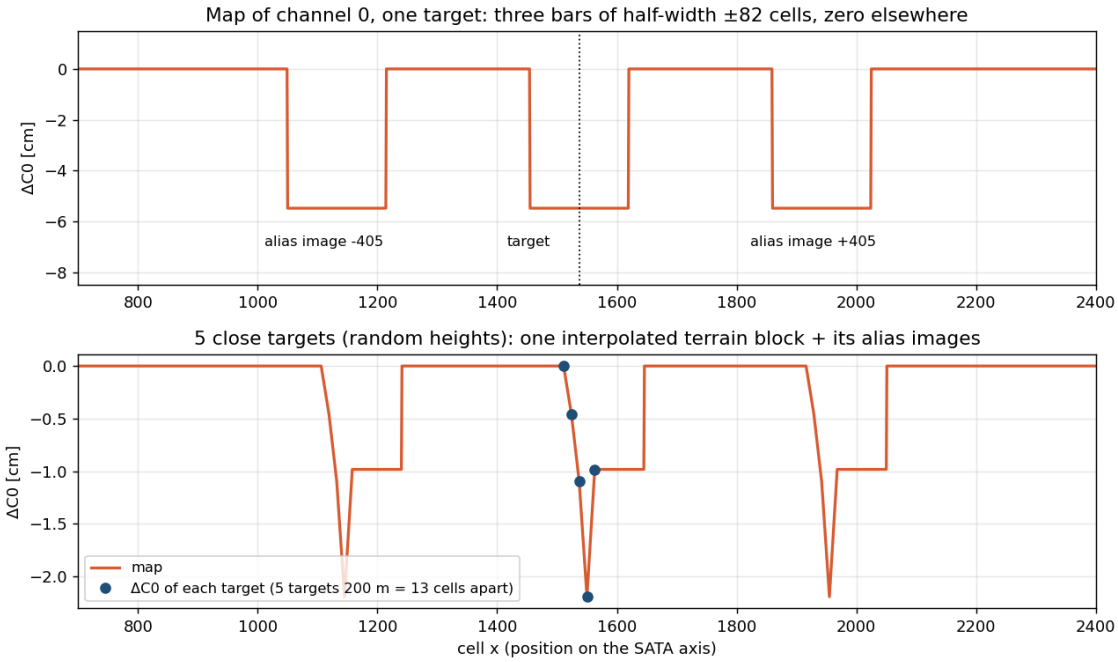
```
def sata_image_offsets(cfg):
    X = r * tan(arcsin(wl*prf/(2*v))) / v * prf      # 405 cells per fold
    B = 4 * v * sin(theta_tx/2) / wl                 # 1281 Hz
    m_max = round(0.5 * B / prf)                     # 1
    return [m * X for m in range(-m_max, m_max+1)]   # [-405, 0, +405]
```

4.5 A5 — Draw the map

We now have, for each target: the value (ΔC_0, A1), the centre (A2), the width (A3) and the shifts (A4). The map starts all at zero (flat ground) and receives:

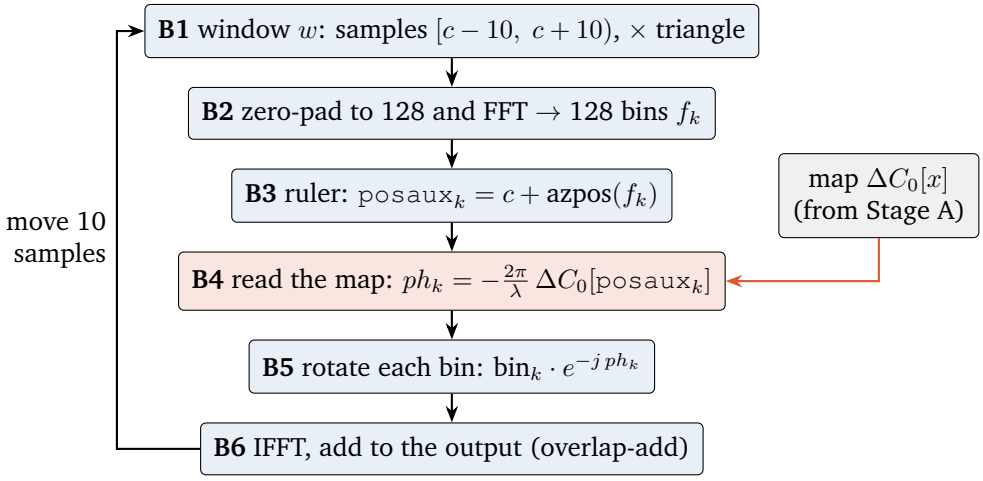
1. **Direct layer.** The targets are sorted by pixel. Neighbouring targets whose regions touch (distance $\leq 2hw$) form a **terrain block**: from (first $- hw$) to (last $+ hw$), each cell gets the ΔC_0 **interpolated** between the neighbouring targets (like reading a DEM). An isolated target is a block on its own: constant ΔC_0 over $x_{\text{target}} \pm hw$.
2. **Alias layers.** The direct layer is copied shifted by -405 and $+405$ cells, **only where the direct layer does not already cover**.
3. Everything else stays **zero**.

```
def footprint_map(values_by_pixel, naz, hw, offsets):
    arr = zeros(naz); cov = zeros(naz, bool)
    for block in blocks(pixels, gap <= 2*hw):              # 1) direct layer
        x = range(block[0]-hw, block[-1]+hw+1)
        arr[x] = np.interp(x, block_pixels, block_values); cov[x] = True
    for off in (-X, +X):                                   # 2) alias layers
        arr[where the shifted copy exists and ~cov] = shifted copy
    return arr                                             # 3) rest = 0
```



Top: the map of the example (1 target, channel 0): -5.474 cm over 1537 ± 82, 1132 ± 82 and 1942 ± 82. Bottom: 5 targets 200 m apart (13 cells): they become one block, interpolated between the values of each target, plus the alias copies.

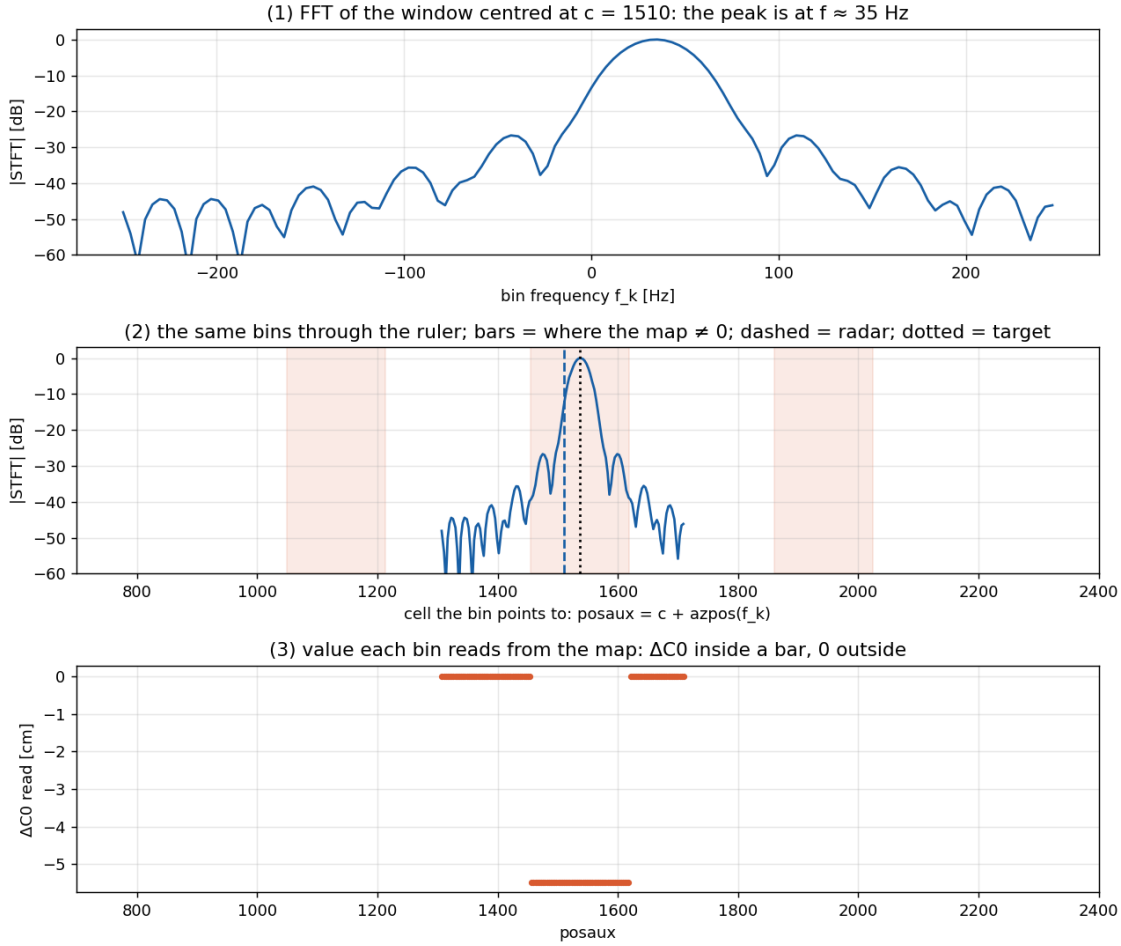
5 Stage B — The loop (once per window)



Where this happens in the code: `sata_1d` in `sar_recon/sata.py`

```
Tsubeff, hop, Nzp = 20, 10, 128                          # window, hop, FFT size
for start in range(0, dimx, hop):                         # ~307 windows per
    channel
    seg  = data[start:start+Tsubeff]                      # B1
    buf  = zeros(Nzp); buf[:len(seg)] = seg * win         # B1 (triangle)
    spec = np.fft.fft(buf)                                # B2
    center = start + 0.5*Tsubeff                          # radar position
    posaux = np.clip(np.round(azpos + center), 0, dimx-1)    # B3
    ph   = -2*np.pi/wl * delta_C0_array[posaux]           # B4  <-- the map is
    used HERE
    spec = spec * np.exp(-1j*ph)                          # B5  (inverse=True:
    remove)
    out[start:start+len(seg)]  += np.fft.ifft(spec)[:len(seg)]    # B6
    wsum[start:start+len(seg)] += win[:len(seg)]
out = out / wsum
```

5.1 One window, with numbers (centre $c = 1510$)



1. **B1–B2.** The FFT of the 20 samples around 1510 has a peak at $f \approx +35$ Hz (panel 1). That is the target: the radar is 27 cells before it, approaching slowly.
2. **B3.** Through the ruler, azpos(35 Hz) $\approx +28$ cells $\Rightarrow$ `posaux` $= 1510 + 28 =$ **1538** (panel 2): the peak lands **on the target** (dotted line), inside the central bar.

3. **B4.** These bins read $\Delta C_0[1538] = -5.474$ cm (panel 3). The phase:

$$ph = -\frac{2\pi}{\lambda}\Delta C_0 = -\frac{2\pi}{0.25}\,(-0.05474) = +1.376 \text{ rad} = +78.8.$$

   The bins that point outside the bars read 0 $\Rightarrow ph = 0$.

4. **B5.** Each bin is multiplied by $e^{-j\,ph}$: the target's bins are rotated by -78.8; the others do not change.

Why does rotating by -78.8 correct it?

The simulator records $e^{-j2\pi\, rh/\lambda}$. The extra path ΔC_0 adds $e^{-j2\pi\Delta C_0/\lambda} = e^{-j2\pi(-0.05474)/0.25} = e^{+j\,78.8}$ to the target's signal. Multiplying by $e^{-j\,78.8}$ cancels exactly that.

5.2 The same target across the windows

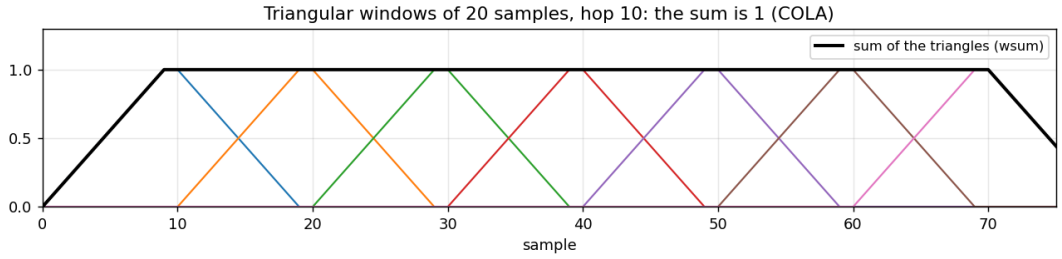
Each window the radar moves 10 samples, the target's Doppler changes and the peak moves to another bin. But that other bin points to the same place (or to an alias image), and reads the same ΔC_0:

centre c	true Doppler	recorded	peak at f_k	azpos	`posaux`	ΔC_0 read [cm]	landed on
1110	+527	+27	+27	+22	1132	−5.47	alias −1
1210	+404	−96	−94	−76	1134	−5.47	alias −1
1310	+281	−219	−219	−177	1133	−5.47	alias −1
1410	+157	+157	+156	+127	1537	−5.47	direct
1510	+34	+34	+35	+28	1538	−5.47	direct
1610	−90	−90	−90	−73	1537	−5.47	direct
1710	−213	−213	−211	−171	1539	−5.47	direct
1810	−337	+163	+164	+133	1943	−5.47	alias +1
1910	−460	+40	+39	+32	1942	−5.47	alias +1
2010	−583	−83	−82	−66	1944	−5.47	alias +1

Reading one row: in window 1110 the true Doppler is +527 Hz, the channel records +27 Hz (folded), the ruler sends it to $1110 + 22 = 1132 = 1537 - 405$, which is the alias bar — and the bin gets the target's ΔC_0. In the middle windows the target lands directly on ≈ 1537. In the last ones, on the $+405$ bar. **In every window, the bin that holds the target gets the same -5.474 cm.**

5.3 B6 — Putting the windows back together (overlap-add)

After the rotation, each window goes back to time (IFFT) and is added to the output. The windows overlap by 50% and each was multiplied by a triangle; the sum of the triangles is 1 everywhere (the COLA condition), so without correction ($\Delta C_0 = 0$) the output equals the input.



As a formula, the output at sample n is

$$s_{\text{out}}[n] = \frac{\sum_w \text{IFFT}\Big\{\ \text{FFT}\{\text{win}\cdot s\}_w[k]\ \cdot\ e^{-j\,ph_{w,k}}\Big\}[n]}{\sum_w \text{win}_w[n]}, \qquad ph_{w,k} = -\frac{2\pi}{\lambda}\,\Delta C_0\big[\,c_w + \text{azpos}(f_k)\,\big].$$

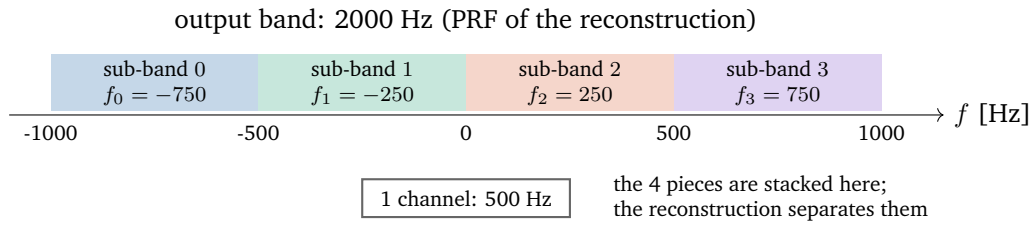
Where ΔC_0 is applied along the positions

The map index is $c_w + \text{azpos}(f_k)$: **where the radar is** (the window) **plus where the bin points** (the frequency). The same ground cell is read by different bins in different windows — exactly the bins in which that piece of ground appears. That is why the correction "follows" each target through the whole aperture.

6 Stage C — The reconstruction (what happens next)

We need it to understand the difference between whole band and per sub-band.

Each channel, sampled at 500 Hz, has a 500 Hz spectrum in which 4 pieces of the true 2000 Hz band are **stacked** (the folded pieces). The reconstruction uses the 4 channels to "unstack" them: for each channel frequency φ it solves a 4×4 system and recovers the 4 pieces, which become the 4 **sub-bands** of the output:

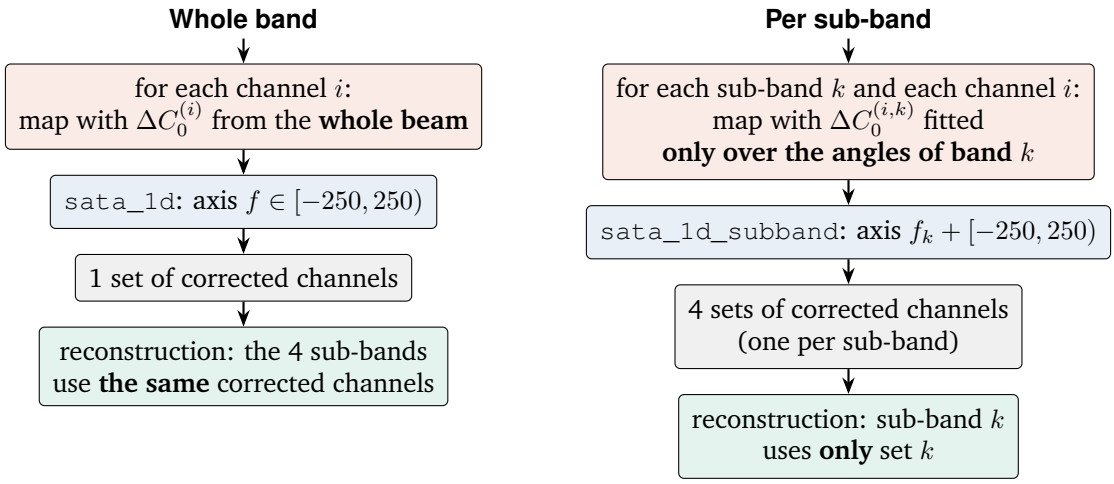


$$f_k = \Big(-\frac{N_{rx}}{2} + k + \frac{1}{2}\Big)\text{PRF}_{op} = -750,\ -250,\ +250,\ +750 \text{ Hz}, \qquad \hat{S}_k(\varphi) = \sum_{i=0}^{3} H^{-1}_{k,i}(\varphi)\ S_i(\varphi).$$

This only works if the phase of each channel is what the filter H^{-1} expects — that is why SATA corrects the channels **before**.

7 Whole band × per sub-band

7.1 The two flows




```
# per sub-band  (sar_recon/subband_sata_explicit.py:
   reconstruct_subband_explicit)
for k in range(Nrx):                                          # each output sub-
   band
    for i in range(Nrx):                                      # each channel
        mapa  = build_delta_term_subband_array(cfg, tr, i, k)     # STAGE A,
   band-k version
        corr  = sata_1d_subband(ch[i], mapa, f_k=f_k, beta_k=beta_k)    # STAGE
   B, band-k axis
        spec[k][i] = fft(corr)
    srec[band k] = sum_i spec[k][i] * iHf[band k, i]          # STAGE C with set
     k only
```

That is 4 kernel runs per channel (16 in total), against 4 in the whole band.

7.2 The three differences, one by one

Difference 1 — ΔC_0 is fitted per band (Stage A1). In the whole band, the parabola of Step A1 is fitted over the *whole* aperture. In the sub-band path, the function `getcoeff_beam` uses only the stretch where the look angle is inside band k:

$$\theta \in [\beta_{lo},\ \beta_{hi}], \qquad \beta = \arcsin \frac{\lambda(f_k \mp \mathrm{PRF}_{op}/2)}{2v}.$$

band	f_k	β_k	angle window	part of the aperture used
0	-750 Hz	-0.699	$[-0.932,\ -0.466]$	radar already past the target
1	-250 Hz	-0.233	$[-0.466,\ 0]$	just after closest approach
2	$+250$ Hz	$+0.233$	$[0,\ +0.466]$	just before
3	$+750$ Hz	$+0.699$	$[+0.466,\ +0.932]$	radar still far, approaching

If the extra path depended on the look angle, each band would get a different value.

Difference 2 — the frequency axis of the kernel (Stage B2–B3). In the sub-band path each bin is labelled with the *absolute* frequency of the band, and the ruler is measured from the central beam of the band:

$$f^{(k)} = f_k + \varphi, \qquad \mathrm{azpos}^{(k)}(\varphi) = \frac{r\left[\tan\beta(f_k+\varphi) - \tan\beta_k\right]}{v}\,\mathrm{PRF}_{op}, \qquad \varphi \in [-250, 250)\ \text{Hz}.$$

Difference 3 — who uses the result (Stage C). In the whole band, the 4 output sub-bands are computed from the *same* set of corrected channels. In the sub-band path, band k uses the set corrected *for it*.

7.3 Why, here, both give the same result

(a) ΔC_0 is the same in every band. For this geometry (iso-range target, broadside), the extra path does not depend on the angle:

channel	whole band	band 0	band 1	band 2	band 3
0	-5.474	-5.474	-5.474	-5.474	-5.474 cm
1	-2.319	-2.319	-2.319	-2.319	-2.319 cm
2	-0.352	-0.352	-0.352	-0.352	-0.352 cm
3	-0.142	-0.142	-0.142	-0.142	-0.142 cm

(b) The band's ruler points to the same cells. With small angles, $\tan\beta \approx \lambda f/2v$, and then, step by step:

$$\text{azpos}^{(k)}(\varphi) \approx \frac{r\,\text{PRF}_{op}}{v}\left[\frac{\lambda(f_k+\varphi)}{2v} - \frac{\lambda f_k}{2v}\right] = \frac{r\,\text{PRF}_{op}}{v}\cdot\frac{\lambda\,\varphi}{2v} \approx \text{azpos}_{\text{whole}}(\varphi).$$

f_k **cancels out**: the same bin (the same recorded frequency φ) points to the same cell in both kernels. Checked numerically: in windows 1110, 1510 and 1910, the target lands on 1132 / 1538 / 1942 **in all 4 bands**, same as the whole band. That is why the map of every band uses the same bars (1537 ± 82, ± 405).

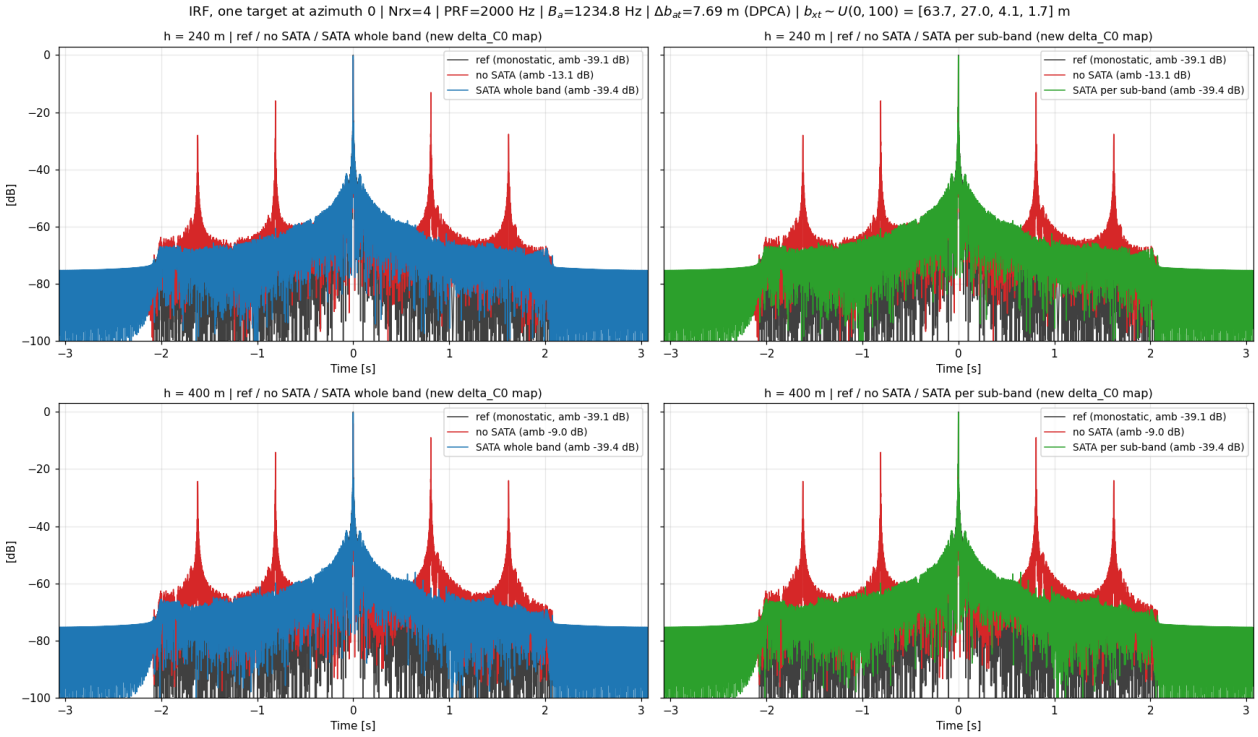
Conclusion

Same ΔC_0 + same cells $\Rightarrow$ the 4 sub-band runs apply the same correction as the single whole-band run. Result: ambiguity -39.4 dB for both (1-target IRF) and a difference ≤ 0.02 dB over all 362 cases of the sweep.

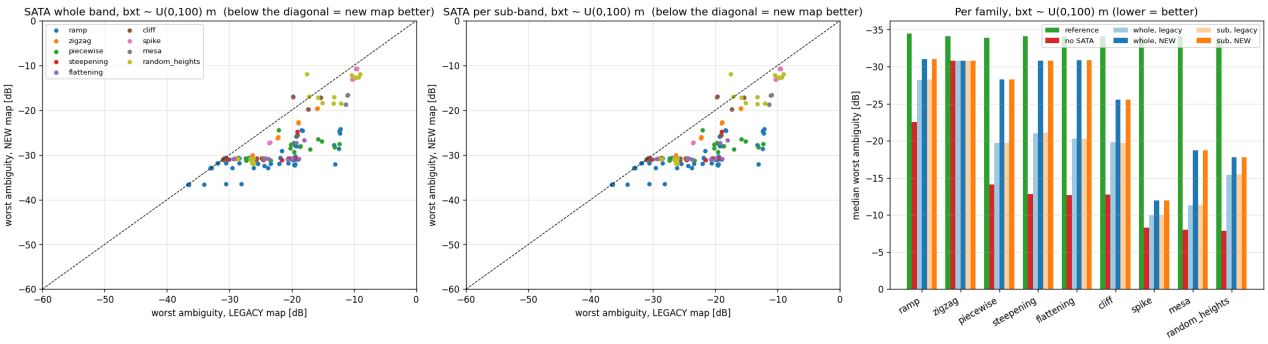
7.4 When per sub-band would make a difference

- **C1/C2 corrections**: these terms use the *absolute* frequency of the bin ($ph \mathrel{-}= 2\pi\,\Delta C_1\,f^{(k)}$), and then f_k does not cancel.
- ΔC_0 **that depends on the angle**: targets off the iso-range surface, squint, geometries in which the extra path changes along the aperture.
- **Large angles**, where $\tan\beta$ is no longer linear.
- **Alias collisions**: separating by band is the natural place to tell the folded energy of one target from the direct energy of another — but that would need different maps per band, which does not happen today.

8 The result



IRF of 1 target (DPCA, $b_{xt} \sim U(0, 100)$). Left: whole band; right: per sub-band. Red: no SATA (-13.1 / -9.0 dB ambiguity). Blue/green: with SATA, -39.4 dB, at the level of the reference (-39.1 dB).



Geometry sweep (181 cases, $b_{xt} \sim U(0, 100)$): points below the diagonal = new map better than the old one. Median: -3.7 dB more suppression; worse in 3 extreme cases (jumps of 500–1000 m between targets 200 m apart).

9 Summary: the order of execution

1. **Simulation**: `getRawData1D` generates the 4 channels $s_i[n]$ (500 Hz, 3072 samples).
2. **For each channel i — Stage A** (`build_delta_C0_array`):
   (a) A1 `residual_C0`: ΔC_0 of each target (parabola fits, target − flat centre).
   (b) A2 `az_pixel_of_scatterer`: cell of each target $(N/2 + (x_0 + dx + b_{at}/2)/\Delta s)$.
   (c) A3 `sata_footprint_halfwidth`: $hw = 82$ (peak width of the window, via the ruler).
   (d) A4 `sata_image_offsets`: $\{-405, 0, +405\}$ (folded energy, via the ruler).

(e) A5 `footprint_map`: interpolated blocks + alias copies; zero elsewhere.

3. **For each channel i — Stage B** (`sata_1d`), for each window: B1 cut, B2 FFT, B3 ruler, **B4 read the map**, B5 rotate, B6 IFFT + add.
4. **Stage C**: reconstruction with the corrected channels.
5. **Per sub-band**: steps 2–3 are repeated for each sub-band k (with the band's ΔC_0 and the axis $f_k + \varphi$), and band k of the reconstruction uses only its own version.

Where each thing lives

function (file)	role
`sata_channels` (`sata.py`)	whole band: Stage A + B for each channel
`build_delta_C0_array` (`sata.py`)	all of Stage A (`footprint` mode; `mode="hold"` = old map)
`residual_C0` (`sata.py`)	A1
`az_pixel_of_scatterer` (`sata.py`)	A2
`sata_footprint_halfwidth` (`sata.py`)	A3
`sata_image_offsets` (`sata.py`)	A4
`footprint_map` (`sata.py`)	A5
`sata_1d` (`sata.py`)	Stage B, whole band (unchanged)
`reconstruct_subband_explicit` (`subband_sata_explicit.py`)	per sub-band: A + B per band, and Stage C per band
`build_delta_term_subband_array` (`subband_recon.py`)	Stage A per band (ΔC_0 over the band's angles)
`sata_1d_subband` (`subband_sata_explicit.py`)	Stage B per band (axis $f_k + \varphi$)